

Горчак Дмитрий, 212. Git: локальная система контроля версий. Назначение, команды, встроенная справка

Система контроля версий хранит историю проекта: не последнюю версию каждого файла, а все состояния, через которые проект прошёл, вместе с объяснением, зачем каждое из них появилось. Нужна она не только там, где над кодом работают несколько человек. Она нужна уже тогда, когда работает один, но дольше одного вечера: папки вида `main_v2_final_испр` не помнят ни порядка изменений, ни их причины, и через неделю нельзя сказать, какая копия рабочая.

На семинаре мы разбирали контроль версий в общем виде: хранилище, получение файла на правку (`check-out`), возврат изменённого файла (`check-in`) и конфигурацию – зафиксированный набор версий, из которых собрана программа. В этом эссе я разбираю, как то же самое устроено в `git`: из чего он состоит, какими командами делается работа, как спросить у него справку и что означают опции команд. Примеры – из моего репозитория `~/cones` с задачей о конусах на плоскости.

Слова, которые дальше встречаются постоянно

Термины `git` звучат непривычно, поэтому сразу коротко объясню, что они означают.

Слово	Что означает
репозиторий	хранилище истории проекта; лежит в скрытой папке <code>.git</code> рядом с самим кодом
коммит	сохранённое состояние всех файлов проекта сразу, с автором, датой и сообщением о том, что изменено
индекс	промежуточная область: сюда командой <code>git add</code> переносят изменения, которые должны войти в следующий коммит
хеш	короткая строка вроде <code>33d77fd</code> , посчитанная по содержимому; служит именем коммита
ветка	подвижный указатель на коммит; позволяет вести отдельную работу, не трогая основную
слияние (<code>merge</code>)	объединение изменений двух веток в одно состояние
тег	неподвижная метка на коммите, например «состояние к 19.09»

Что даёт контроль версий

Система отвечает на вопросы, которые иначе приходится держать в голове: что именно изменилось между двумя состояниями программы; когда и кем сделано изменение и с каким объяснением; как вернуть предыдущее состояние файла или всего проекта; какие версии файлов составляли конкретную рабочую сборку; как объединить работу, которая велась параллельно.

Пока проект небольшой и делается одним человеком, чаще всего нужны первые три возможности: увидеть разницу, вспомнить, зачем была сделана правка, и вернуться к состоянию, которое точно работало.

Почему `git` называют локальной системой

В более старых системах – `CVS`, `Subversion` – история хранится на сервере, и почти любое действие требует связи с ним. В `git` вся история лежит рядом

с кодом, в папке `.git` внутри проекта. Сохранить версию, посмотреть историю, сравнить два состояния, откатиться, завести ветку и слить её обратно можно без сети: всё это происходит на своём компьютере.

Отсюда и порядок работы, принятый в задаче: сначала локальный репозиторий, и только потом облако. Облачный репозиторий – это ещё одна копия того же самого `.git`, и команды при переходе к нему почти не меняются.

Три места, где живёт файл

Главное в устройстве `git` – то, что правка проходит три состояния:

- 1) рабочий каталог – файлы, с которыми вы прямо сейчас работаете в редакторе; здесь изменение ничем не защищено;
- 2) индекс – то, что подготовлено к сохранению командой `git add`; сюда можно положить часть правок, а остальные оставить на потом;
- 3) репозиторий `.git` – сохранённые версии; изменения переносит туда команда `git commit`, и случайной правкой файла их уже не потерять.

```
[рабочий каталог] --git add--> [индекс] --git commit--> [репозиторий .git]
[рабочий каталог] <--git restore-- [индекс] <--git restore --staged-- [.git]
```

Две ступени вместо одной кажутся лишними, пока не понадобится сохранить одно изменение, а второе доделать позже. Команда `git status` показывает, в каком состоянии сейчас каждый файл, и подсказывает команду для следующего шага; её полезно выполнять перед каждым сохранением.

Что хранит коммит

Коммит – не список изменённых строк, а снимок всех файлов проекта на момент сохранения. Кроме самих файлов в нём записаны автор, дата, сообщение и ссылка на предыдущий коммит (у слияния их две). Из этих ссылок и складывается история: каждое состояние знает, из какого оно получилось.

Имя коммита – хеш, то есть короткая строка, вычисленная по его содержимому. Если изменится хотя бы один знак в любом файле, хеш будет другим, поэтому незаметно подменить сохранённую версию нельзя.

Ветка в `git` устроена проще, чем кажется: это указатель, который сдвигается на последний коммит. Поэтому создание ветки почти ничего не стоит. Тег – такой же указатель, только неподвижный; по сути это конфигурация из семинара, то есть зафиксированный состав версий, по которому программу можно собрать заново ровно такой, какой она была в нужный момент.

Установка и первичная настройка

У меня `MSYS2`, поэтому `git` ставится её менеджером пакетов одной командой:

```
$ pacman -S git
$ git --version
git version 2.55.0
```

Перед первым сохранением нужно указать, кто автор изменений: эти данные попадают в каждый коммит. Настройки хранятся на трёх уровнях – для всей машины (`--system`), для пользователя (`--global`) и для отдельного

репозитория (--local), причём более узкий уровень перекрывает более широкий.

```
$ git config --global user.name "Gorchak Dmitriy"
$ git config --global user.email "почта@пример"
$ git config --list --local          # что задано в этом репозитории
```

На Windows пригодится ещё настройка core.autocrlf. Конец строки в Windows и в Linux записывается разными невидимыми символами, и без этой настройки git может показать изменённым весь файл, хотя поменялась одна строка.

Основные команды

Рабочий набор небольшой: примерно десяти команд хватает на всё, что нужно на этом этапе.

Команда	Что делает
git init	создать репозиторий: появляется папка .git
git status	что изменено и что уже подготовлено к сохранению
git add ФАЙЛ	перенести изменения в индекс
git commit -m "текст"	сохранить новую версию с сообщением
git diff	показать правки, которых ещё нет в индексе
git diff --staged	показать то, что войдёт в следующий коммит
git log	история коммитов
git show ХЕШ	что именно изменилось в конкретном коммите
git restore ФАЙЛ	вернуть файл к последней сохранённой версии
git rm, git mv	удалить и переименовать файл, сохранив историю
git branch, git switch	список веток и переход между ними
git merge ВЕТКА	слить работу из другой ветки
git tag	поставить неподвижную метку версии
git remote, git push	отправить историю в облако – следующий этап

Встроенная справка

Справка лежит прямо в программе, и спросить её обычно быстрее, чем искать ответ в интернете. Есть три формы вызова:

- 1) git help КОМАНДА – полное руководство по команде;
- 2) git КОМАНДА --help – то же самое руководство;
- 3) git КОМАНДА -h – короткая справка на несколько строк, только список опций; чаще всего нужна именно она.

Кроме руководств по отдельным командам есть общие разделы. Команда git help -a выводит список всех команд, git help -g – список руководств по темам, а дальше открывается нужное: git help glossary объясняет понятия, git help revisions – как ссылаться на коммиты, git help everyday – типовые приёмы работы.

```
$ git help -a | head -8
$ git commit -h
$ git help revisions
```

В MSYS2 руководство открывается прямо в окне терминала: листается пробелом и стрелками, выход – клавиша q. Незнакомый ключ я сначала смотрю

в справке: это быстрее, чем потом разбирать последствия.

Опции

Опция – уточнение к команде. Короткие опции пишутся одной буквой после дефиса, длинные – словом после двух дефисов; у многих есть обе формы: `-m` и `--message`, `-q` и `--quiet`. Короткие можно склеивать: `git commit -am` это то же, что `git commit -a -m`.

Отдельно стоит знать про разделитель из двух дефисов. Он отделяет опции и имена веток от имён файлов, чтобы `git` не гадал, что имелось в виду, когда ветка и файл называются одинаково.

```
$ git restore -- data/cones.txt
```

```
$ git log -3 -- data/cones.txt      # три последние правки одного файла
```

Сочетания, которыми я пользуюсь чаще всего:

- 1) `git log --oneline --graph --decorate` – история в одну строку на коммит, с рисунком веток и именами меток;
- 2) `git commit -am "текст"` – сохранить правки файлов, которые `git` уже отслеживает, не вызывая `git add` отдельно;
- 3) `git add -p` – просмотреть изменения по кускам и выбрать, что подготовить к сохранению;
- 4) `git diff --staged` – увидеть ровно то, что попадёт в коммит;
- 5) `git commit --amend` – исправить последний коммит, пока он никуда не отправлен.

Что не нужно хранить в репозитории

В репозиторий кладут исходные тексты, данные и описание сборки, но не то, что компилятор и сама программа создают заново. Правило простое: если файл можно получить из того, что уже лежит в репозитории, хранить его не надо – иначе история растёт, а пользы нет. Список исключений записывается в файл `.gitignore`:

```
# результаты сборки
build/
bin/
*.o
*.exe

# файлы, которые создаёт сама программа
cones_plan.txt
cones_field.txt
cones_range.gp

# рисунки gnuplot в корне проекта
/*.png
```

Благодаря этому `git status` показывает чистое рабочее дерево сразу после сборки и запуска программы. Скриншоты для отчёта лежат в папке `essays` и в историю как раз попадают: заново их не получить.

Пример: история задачи о конусах

Репозиторий создан одной командой, а дальше каждый законченный шаг сохранялся отдельно. Классы Point, Circle и Cone попали в разные коммиты, потому что это разные шаги работы: по такой истории видно, в каком порядке собиралась программа, и понятно, где искать причину, если что-то сломалось.

```
$ git init -b main
$ git add include/point.h src/point.cpp
$ git commit -m "Класс Point: точка на плоскости"
```

История репозитория на момент сдачи этапа (читается снизу вверх):

```
* 43a246a (HEAD -> main) Слияние ветки stl-list: работа со списком
|\
| * d5efe6d (stl-list) Двухнаправленный список STL: сортировка
|/
* 33d77fd (tag: stage-19-09) Скрипт gnuplot: вид сверху и ландшафт
* eeefa7df Программа cones_app и сборка: build.sh и CMakeLists.txt
* dc8a93d Чтение свойств из файла, таблица и данные для gnuplot
* 27a018e Класс Cone: конус (x, y, r, h), объём  $V = S \cdot h / 3$ 
* b8af7c1 Класс Circle: круг и площадь как вычисляемый атрибут
* e117716 Класс Point: точка на плоскости, поля private
* f4dfd46 Начало работы: README и .gitignore
```

Две вещи здесь сделаны сознательно. Работа над списком из STL шла в отдельной ветке stl-list и вливалась слиянием с ключом --no-ff: так в истории видно, что это была отдельная задача, а не правка по ходу дела. А на коммит, соответствующий сроку 19.09, поставлена метка stage-19-09 – по ней проект собирается ровно в том состоянии, в каком сдавался.

Обычная правка занимает четыре команды: посмотреть, что изменилось, посмотреть, как именно, подготовить к сохранению и сохранить.

```
$ git status -s
$ git diff
$ git add data/cones.txt
$ git commit -m "Ещё один конус в исходных данных"
```

Ошибки, на которых я учился

- 1) забыть git add – коммит уходит без изменений; помогает привычка смотреть git status перед каждым сохранением;
- 2) сложить в один коммит несколько несвязанных правок – потом не понять, какая из них сломала сборку;
- 3) сообщения вида «fix» и «работает» – через неделю по ним ничего не восстановить; пишу, что и зачем изменено;
- 4) хранить собранные программы и рисунки – репозиторий растёт, а смысла нет;
- 5) не настроить core.autocrlf на Windows – тогда git показывает изменённым весь файл вместо одной строки.

Что дальше: облако

Когда локальная часть освоена, добавляется два шага: указать адрес копии в облаке и отправить туда историю. Остальные команды не меняются, push просто переносит уже сделанные коммиты.

```
$ git remote add origin АДРЕС_РЕПОЗИТОРИЯ  
$ git push -u origin main
```

Заключение

Git хранит не файлы, а историю состояний проекта, и позволяет вернуться в любое из них. Работа с ним сводится к десятку команд и к привычке сохранять законченные шаги отдельно и с понятным сообщением. Точное значение любой опции можно уточнить во встроенной справке, не выходя из терминала.

Главная логика:

```
[правка файла] -(git add)-> [индекс] -(git commit)-> [история в .git]  
[история] -(git log, git show, git diff)-> [что и когда изменилось]  
[история] -(git restore, git checkout)-> [возврат к нужному состоянию]  
[история] -(git push)-> [копия в облаке]
```